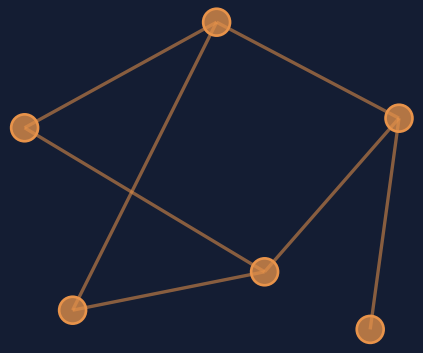
YIĞINLAR VE UYGULAMALARI

4. Hafta — Stack ADT, Dizi/Bağlı Liste Gerçekleştirimi ve Uygulamaları

Karadeniz Teknik Üniversitesi

Yazılım Geliştirme Bölümü

Dr. Öğr. Üyesi Sayyad Alizadeh



Geçen Haftadan Bağlantı

- Bağlı listede başa ekleme ve baştan silmenin O(1) olduğunu gördük.
- struct Dugum ve malloc/free ile dinamik bellek yönetimini öğrendik.
- **Bu hafta bu iki işlemi (başa ekle / baştan sil) YENİ bir isimle yeniden keşfedeceğiz: push ve pop.**

Bu Hafta Neden Önemli?

Yığın (stack), en basit ama en sık kullanılan veri yapılarından biridir — derleyicilerden tarayıcılara kadar her yerde çalışır.

Hem diziyle hem bağlı listeyle gerçekleştirebileceğimizi göreceğiz — iki haftalık bilginiz bir araya gelecek.

Bu Haftanın Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler...

1. Stack ADT'yi ve LIFO prensibini açıklayabilir
2. Diziyle ve bağlı listeyle stack gerçekleştirebilir; push/pop/peek yazabilir
3. Overflow ve underflow durumlarını doğru şekilde ele alabilir
4. Dengeli parantez kontrolü yapan bir algoritma yazabilir
5. Infix bir ifadeyi postfix'e çevirebilir ve postfix ifadeyi stack ile hesaplayabilir
6. Stack ile fonksiyon çağrıları (call stack) arasındaki ilişkiyi açıklayabilir

Bu Haftanın Yol Haritası

STACK TEMELLERİ

ADT, LIFO, dizi ve bağlı liste ile gerçekleştirim

Slayt 5–34

UYGULAMALAR

Parantez kontrolü, infix→postfix, postfix hesaplama

Slayt 35–58

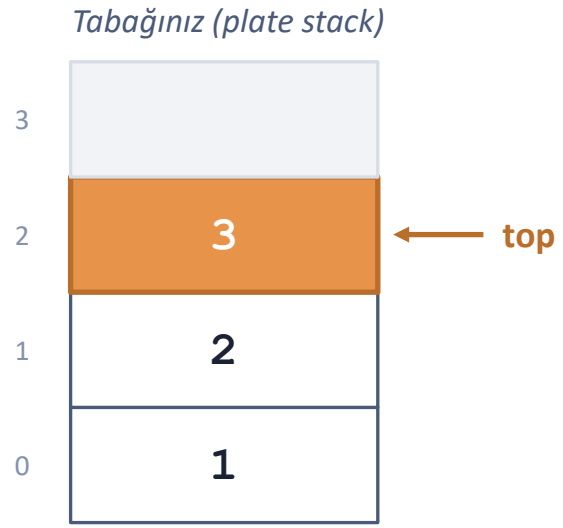
ÖZYİNELEMEYE KÖPRÜ + KAPANIŞ

Call stack, Hanoi motivasyonu, özet, quiz, ödev

Slayt 59–65

Stack (Yığın) Nedir?

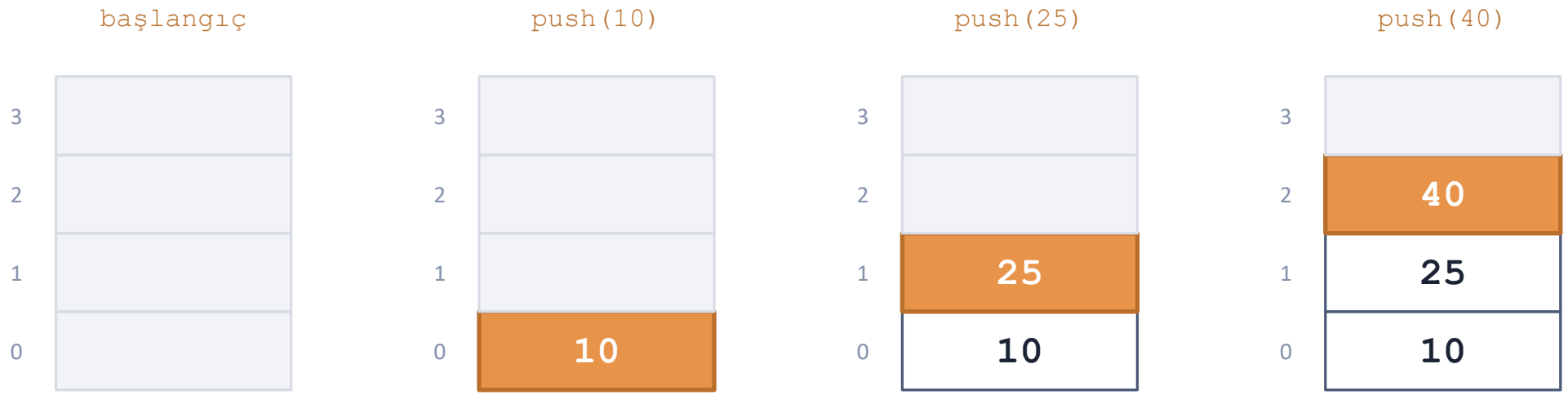
Elemanların yalnızca TEK bir uçtan eklenip çıkarılabildiği, sıralı bir veri yapısıdır.



- Analoji: bir tabak yığını. Yeni tabakı sadece EN ÜSTE koyabilirsiniz, ve sadece EN ÜSTTEKİNİ alabilirsiniz.
- **Ortadaki veya alttaki bir elemana DOĞRUDAN erişim YOKTUR.**

LIFO Çalışma Prensibi

Last In, First Out — Son Giren İlk Çıkar



Sıra: push(10) → push(25) → push(40) → pop() = 40 → pop() = 25 → pop() = 10

En son eklenen eleman ilk çıkar: LIFO. Kuyruklarda ise ilk eklenen eleman ilk çıkar: FIFO. Her iki yapı da diziler veya bağlı listeler kullanılarak gerçekleştirilebilir. Kuyrukları 6. haftada inceleyeceğiz.

Gerçek Hayat ve Yazılımda Kullanım Alanları

Günlük Hayattan

- Tabağ/kitap yığını
- Geri al (Ctrl+Z) işlemleri
- Tarıyıcıda “geri git” tuşu
- Bir yığın tepsi/kutu istifleme

Yazılımda

- **Fonksiyon çağrı yığını (call stack)**
- **İfade değerlendirme (bu haftanın ana konusu)**
- Derleyicilerde parantez/sözdizimi kontrolü
- Özyineleme (recursion) — 5. hafta

Bu haftanın geri kalanı, sağdaki listenin ilk ikisini derinlemesine işleyecek.

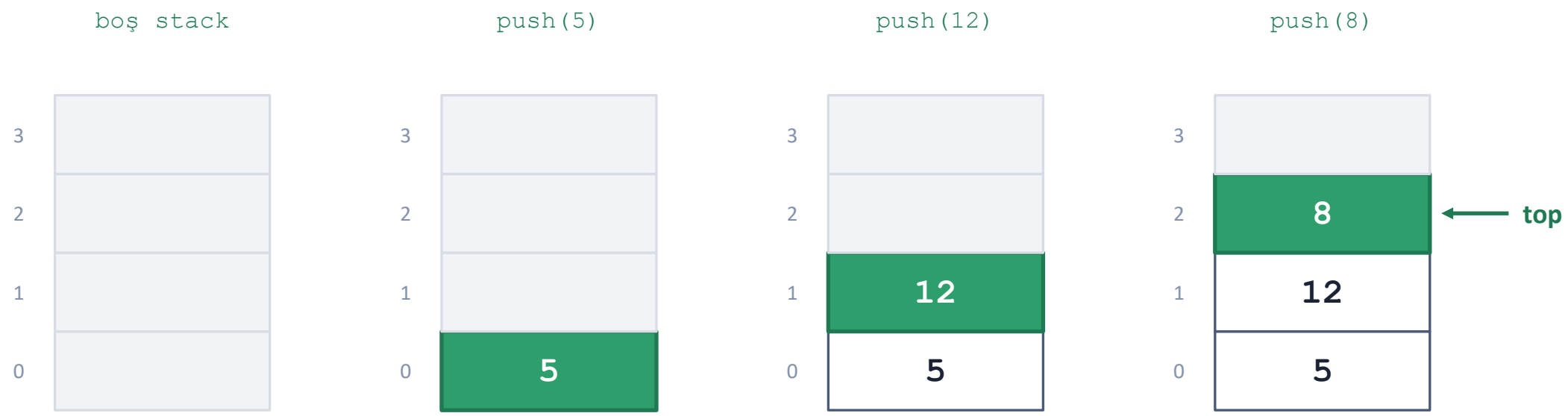
Stack ADT (Abstract Data Type) ve Temel İşlemler

İşlem	Anlamı
`push(x)`	x elemanını stack'in EN ÜSTÜNE ekler
`pop()`	Stack'in EN ÜSTTEKİ elemanını çıkarıp döndürür
`peek() / top()`	En üstteki elemanı çıkarMADAN görüntüler
`isEmpty()`	Stack boş mu diye kontrol eder
`isFull()`	Stack dolu mu diye kontrol eder (sadece dizi tabanlıda anlamlı)

Tüm bu işlemler, doğru gerçekleştirildiğinde O(1) karmaşıklığa sahiptir — bunu ilerleyen slaytlarda kanıtlayacağız.

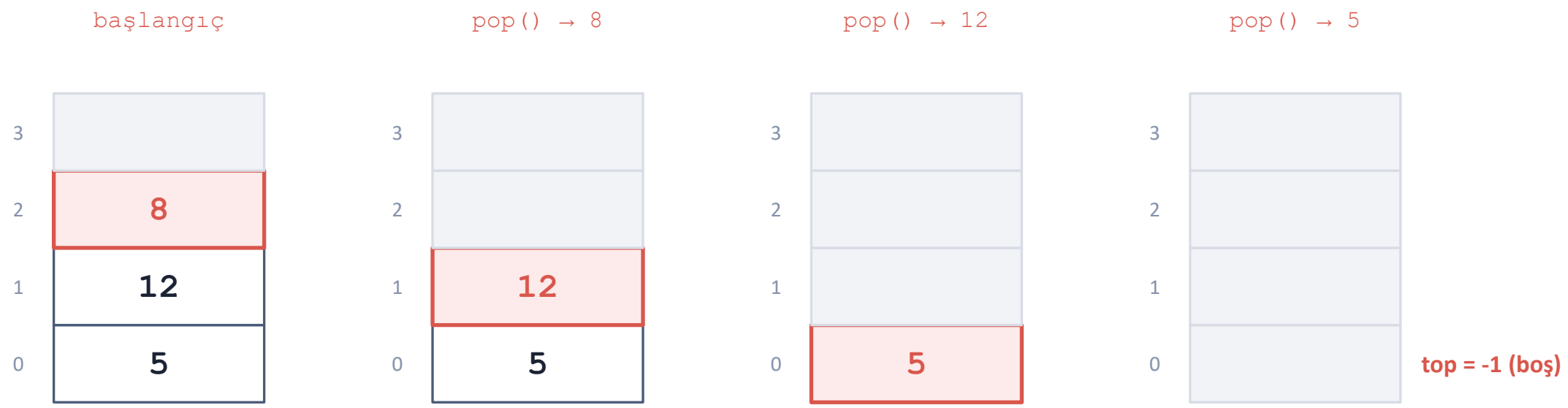
Soyut Veri Tipi (Abstract Data Type)

Push İşlemi — Kavramsal Görsel



Her push işleminde top BİR ARTAR ve yeni eleman o konuma yazılır. Eleman her zaman EN ÜSTE eklenir — başka hiçbir yere değil.

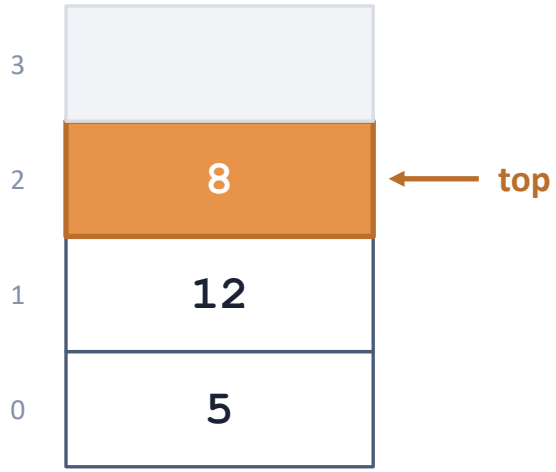
Pop İşlemi — Kavramsal Görsel



Her pop işleminde EN ÜSTTEKİ eleman çıkarılır ve döndürülür, sonra top BİR AZALIR. Çıkan değerlerin sırasına (8, 12, 5) dikkat edin — tam olarak eklenme sırasının TERSİ.

Peek / Top İşlemi

- **peek(); en üstteki elemanı SADECE OKUR, stack'ten çıkarmaz — stack değişmez.**

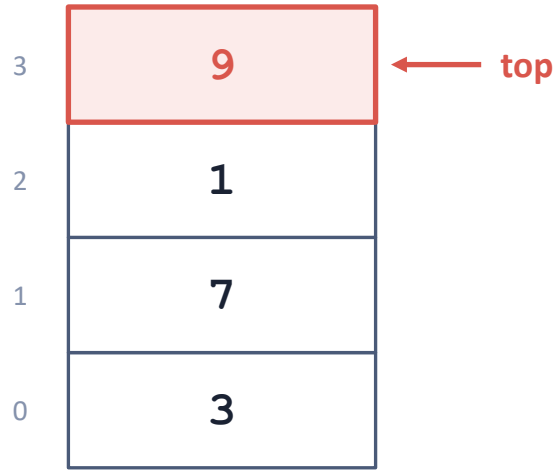



```
peek() → 8   (döner ama silinmez)
```

Kullanım Alanı: pop() yapmadan “sıradaki eleman ne?” diye bakmak istediğinizde kullanılır — örneğin bu haftaki postfix değerlendirme ve parantez kontrolü algoritmalarında sıkça işe yarayacak.

Stack Overflow Kavramı

- **Stack DOLUYKEN push() çağrılmaya çalışılırsa “stack overflow” hatası oluşur.**




```
push(6) → HATA! Kapasite=4, zaten dolu.
```

Sadece DİZİ tabanlı stack'lerde gerçekleşir (sabit kapasite nedeniyle). Bağlı liste tabanlı stack'te, bellek yeterli olduğu sürece overflow oluşmaz.

Stack Underflow Kavramı

- **Stack BOŞKEN pop() veya peek() çağrılmaya çalışılırsa “stack underflow” hatası oluşur.**

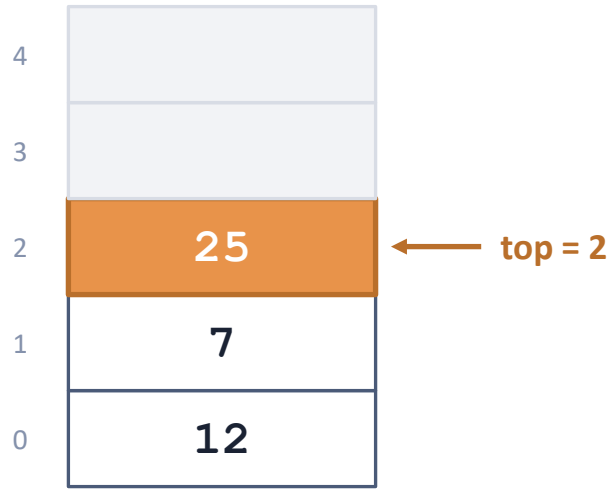
top = -1 (boş)

```
pop() → HATA! Çıkarılacak eleman yok.
```

Hem dizi hem bağlı liste tabanlı stack'lerde geçerlidir. Her pop/peek öncesi isEmpty() kontrolü yapılmalıdır — bu haftaki tüm C kodlarımızda göreceğiz.

Dizi ile Stack Gerçekleştirimi — Kavram

- Sabit boyutlu bir dizi + bir “top” değişkeni ile stack gerçekleştirilebilir.
- **top; dizinin en üstteki DOLU hücresinin İNDEKSİNİ tutar.**



Kapasite (MAX) = 5, ama sadece 3 hücre dolu (top=2). Kalan 2 hücre (3,4) henüz kullanılmıyor.

C’de Stack Yapısı

```
#define MAX 100

struct Stack {
    int veri[MAX];   // elemanları tutan dizi
    int top;         // en üstteki dolu hücrenin indeksi
};

void baslat(struct Stack *s) {
    s->top = -1;     // -1: stack boş
}
```

top = -1, boş stack'i temsil eder. İlk push'ta top = 0 olur ve dizinin 0. hücresine yazılır.

isEmpty / isFull — Pseudocode ve C Kodu

Pseudocode

```
Fonksiyon isEmpty(s)
    dön (s.top == -1)

Fonksiyon isFull(s)
    dön (s.top == MAX - 1)
```

C Kodu

```
int isEmpty(struct Stack *s) {
    return (s->top == -1);
}

int isFull(struct Stack *s) {
    return (s->top == MAX - 1);
}
```

Bu iki fonksiyon, push ve pop'un İÇİNDE her zaman önce çağrılır — overflow/underflow'u önlemenin tek güvenilir yolu budur. Kendi başına karışık bir mantık gerektirmezler, ama atlanmaları en yaygın programlama hatalarından biridir.

Push — Pseudocode

```
Fonksiyon Push(s, deger)
    eğer isFull(s)
        dön HATA   // stack overflow
    s.top ← s.top + 1
    s.veri[s.top] ← deger
```

SIRA ÖNEMLİ: önce isFull() kontrolü yapılır, SONRA top arttırılır, EN SON değer yazılır. Kontrol atlanırsa dizinin sınırları dışına yazılabilir (buffer overflow).

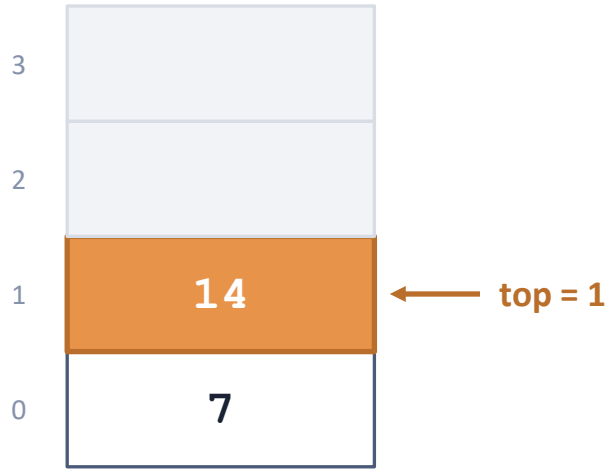
Push — C Kodu

```
int push(struct Stack *s, int deger) {
    if (isFull(s)) {
        return -1;          // stack overflow
    }
    s->top++;
    s->veri[s->top] = deger;
    return 0;               // başarılı
}
```

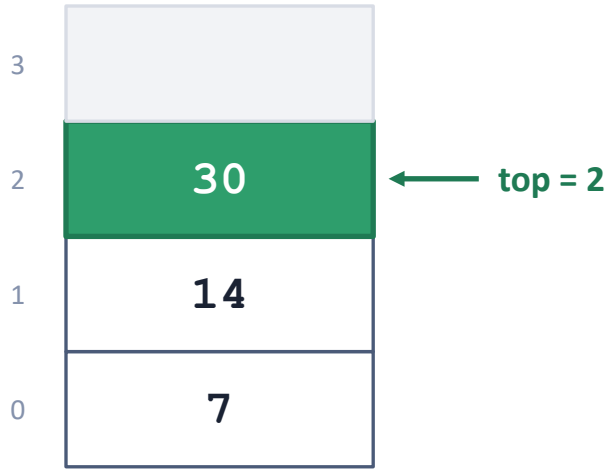
Push — Görsel Adım Adım

push(30) çağrılıyor, kapasite=4

Önce



Sonra: isFull()? Hayır (top=1≠3) → top←2, veri[2]←30



Kodda tam olarak: s->top++ (1→2), sonra s->veri[2] = 30. Sonuç: top=2, veri[2]=30.

Pop — Pseudocode

```
Fonksiyon Pop(s)
    eğer isEmpty(s)
        dön HATA   // stack underflow
    deger ← s.veri[s.top]
    s.top ← s.top - 1
    dön deger
```

SIRA ÖNEMLİ: önce isEmpty() kontrolü, SONRA değer okunur, EN SON top azaltılır. Değer okumadan top'u azaltsak, o değere bir daha erişemezdik.

Pop — C Kodu

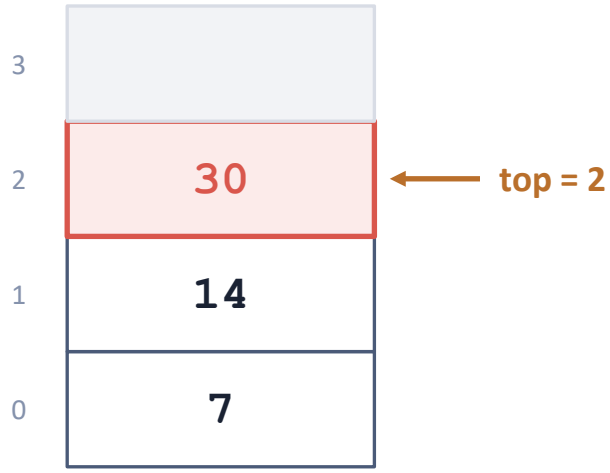
```
int pop(struct Stack *s, int *deger) {
    if (isEmpty(s)) {
        return -1;              // stack underflow
    }
    *deger = s->veri[s->top];  // önce oku
    s->top--;                  // sonra azalt
    return 0;                   // başarılı
}
```

*Neden *deger (işaretçi) kullanılıyor? Çünkü fonksiyon hem BAŞARI/HATA durumunu (return değeri) hem de ÇIKAN DEĞERİ (*deger aracılığıyla) aynı anda döndürmelidir.*

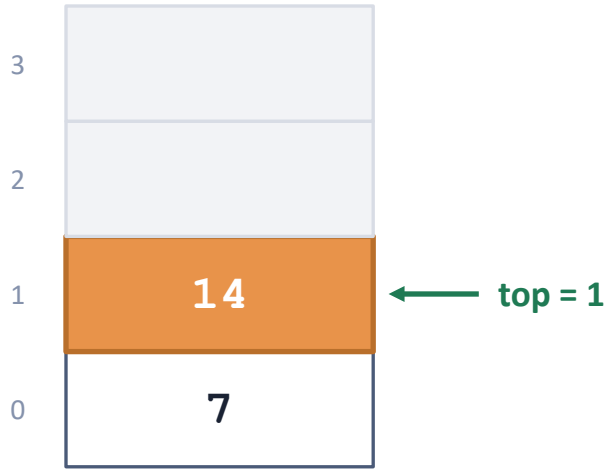
Pop — Görsel Adım Adım

pop() çağrılıyor

Önce



Sonra: *deger←30 (okundu), sonra top←1



Dönen değer: 30. Dizideki 30 değeri fiziksel olarak SİLİNMEZ, sadece top azaldığı için artık "erişilemez" sayılır.

Peek — Pseudocode ve C Kodu

Pseudocode

```
Fonksiyon Peek(s)
    eğer isEmpty(s)
        dön HATA
    dön s.veri[s.top]
```

C Kodu

```c
int peek(struct Stack *s, int *deger) {
    if (isEmpty(s)) {
        return -1;
    }
    *deger = s->veri[s->top];
    return 0;
}
```

pop() ile TEK farkı: s->top-- satırı YOKTUR. Değer okunur ama top değişmez.

Overflow / Underflow Kontrolü — Özet

- Gördüğümüz gibi, her push/pop/peek fonksiyonu KENDİ İÇİNDE isFull()/isEmpty() kontrolü yapar.
- **Bu kontrol asla çağrı yapan kodun sorumluluğuna bırakılmamalıdır.**

Durum	Kontrol	Sonuç
Push, stack doluyken	isFull(s)	return -1 (overflow)
Pop/Peek, stack boşken	isEmpty(s)	return -1 (underflow)

Çağıran kod, push/pop/peek'in return değerini HER ZAMAN kontrol etmelidir (ör. if (push(s,5) == -1) { ... }) — aksi halde hatalar sessizce gözden kaçar.

Dizi Tabanlı Stack — Karmaşıklık Analizi

İşlem	Zaman	Alan
`push`	O(1)	O(1) ek alan
`pop`	O(1)	O(1) ek alan
`peek`	O(1)	O(1) ek alan
`isEmpty / isFull`	O(1)	O(1) ek alan

Tüm işlemler İNDEKSLİ erişim kullandığı için (dizi[top]) sabit zamanlıdır — hiçbir işlemde döngü yoktur. Toplam alan karmaşıklığı ise dizinin kapasitesine (MAX) bağlı olarak O(n)'dir, ama BU BELLEK BAŞTAN AYRILMIŞTIR.

Bağlı Liste ile Stack — Kavram

- **3. haftadan tanıdık struct Dugum'u kullanabiliriz — hiçbir yeni yapı gerekmez!**
- TEK farklılık: head işaretçisi artık “top” olarak adlandırılır — stack'in en üst elemanını gösterir.



En üstteki eleman artık listenin BAŞINDA (head'de) tutulur — çünkü başa ekleme/silme O(1) olduğu için LIFO davranışına mükemmel uyar.

Push = Başa Ekleme — Pseudocode ve C Kodu

Pseudocode

```
Fonksiyon Push(top, deger)    // top: adres (**)
    yeni ← YeniDugum(deger)
    eğer yeni == NULL
        dön HATA          // *top DEĞİŞMEZ
    yeni.sonraki ← *top
    *top ← yeni
    dön BAŞARILI
```

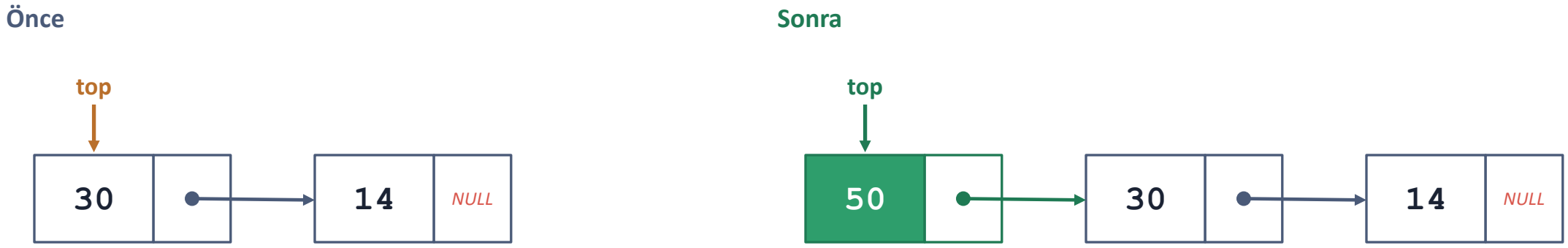
C Kodu

```
int push(struct Dugum **top, int deger) {
    struct Dugum *yeni = yeniDugum(deger);
    if (yeni == NULL) {
        return -1;    // bellek yetersiz
    }
    yeni->sonraki = *top;
    *top = yeni;
    return 0;         // başarılı
}
```

int döndürüp başarı/hata durumunu AYRI bildiriyoruz; struct Dugum **top ile top değişkeni doğrudan güncellenir. Çağrı: push(&top, 5);. Karmaşıklık: O(1).

Push — Görsel (Bağlı Liste)

push(50) çağrılıyor



yeni->sonraki, ESKİ top'u (30) gösterir; SONRA top yeni düğümü (50) gösterir. Bu, 3. haftadaki başa ekleme diyagramlarının TIPKISI — sadece isim değişti.

Pop = Baştan Silme — Pseudocode ve C Kodu

Pseudocode

```
Fonksiyon Pop(top, deger) // top: **, deger: *
    eğer *top == NULL
        dön HATA          // stack underflow
    gecici ← *top
    *deger ← gecici.veri
    *top ← gecici.sonraki
    free(gecici)
    dön BAŞARILI
```

C Kodu

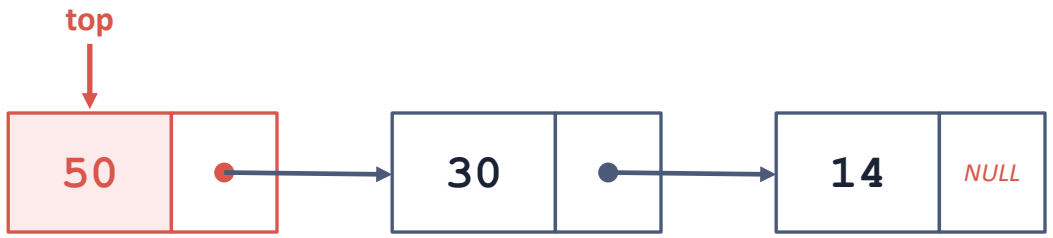
```
int pop(struct Dugum **top, int *deger) {
    if (*top == NULL) {
        return -1;    // stack underflow
    }
    struct Dugum *gecici = *top;
    *deger = gecici->veri;
    *top = gecici->sonraki;
    free(gecici);
    return 0;         // başarılı
}
```

Artık pop, başarı/hata durumunu int ile, çıkan değeri *deger ile ayrı bildiriyor — son düğüm çıkarıldığında *top=NULL olmasıyla “stack boş” hatası ARTIK KARIŞMAZ.

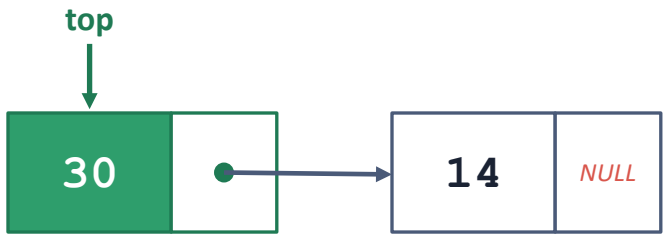
Pop — Görsel (Bağlı Liste)

pop() çağrılıyor

Önce



Sonra: *deger←50 (okundu)



Dönen değer: 50. Eski top düğümü (50) free edildi; top artık bir sonraki düğümü (30) gösteriyor. Karmaşıklık: O(1) — dizi tabanlı versiyonla aynı!

Peek — C Kodu (Bağlı Liste)

```
int peek(struct Dugum *top, int *deger) {
    if (top == NULL) {
        return -1;          // stack underflow
    }
    *deger = top->veri;     // sadece oku
    return 0;
}
```

Yine pop() ile TEK farkı: top değişkeni İLERLETİLMEZ ve free() çağrılmaz. Sadece en üstteki düğümün verisi okunur.

Bellek Güvenliği Hatırlatması

- **Bağlı liste tabanlı stack, 3. haftadaki tüm bellek kurallarını aynen taşır.**
- push() başarısız olabilir (malloc NULL dönerse) — mutlaka kontrol edilmeli.
- pop() sonrası free edilen düğüm bir daha kullanılmamalı (sarkan işaretçi riski).
- Stack artık gerekmiyorsa, içindeki TÜM düğümler döngüyle pop edilip free edilmelidir — aksi halde bellek sızıntısı oluşur.

Dizi tabanlı stack'te bu riskler YOKTUR — bellek program başında bir kez ayrılır ve program sonunda otomatik geri alınır.

Dizi vs Bağlı Liste Stack — Karşılaştırma

Özellik	Dizi Tabanlı	Bağlı Liste Tabanlı
push/pop/peek	O(1)	O(1)
Overflow riski	VAR (sabit kapasite)	YOK (bellek yeterse)
Bellek fazlası	Yok	Her düğümde 1 işaretçi
Boyut esnekliği	Sabit (MAX)	Dinamik
Uygulama basitliği	Biraz daha basit	malloc/free yönetimi gerekir

Pratikte: girdi boyutu önceden biliniyorsa dizi tabanlı tercih edilir (daha hızlı, daha basit); belirsizse bağlı liste tabanlı güvenlidir.

Dengeli Parantez Problemi

Uygulama 1

- Bir ifadedeki (, [, { parantezlerinin DOĞRU SIRAYLA ve EŞİT SAYIDA kapandığını kontrol etmek istiyoruz.

`{ ( [ ] ) }`	✓ Dengeli
`{ ( ] ) }`	✗ Dengesiz
`( (a+b)`	✗ Dengesiz

Bu, derleyicilerin kodunuzu okurken sürekli yaptığı bir kontroldür — ve stack ile şaşırtıcı derecede basit çözülür.

Parantez Kontrolü — Pseudocode

```
Fonksiyon DengeliMi(ifade)
    s ← BoşStack()
    her karakter c için ifade'de
        eğer c açan parantez ise  ( [ {
            push(s, c)
        eğer c kapayan parantez ise  ) ] }
            eğer isEmpty(s)  dön YANLIŞ
            ust ← pop(s)
            eğer ust, c ile eşleşmiyorsa  dön YANLIŞ
    dön isEmpty(s)
```

Parantez Kontrolü — C Kodu

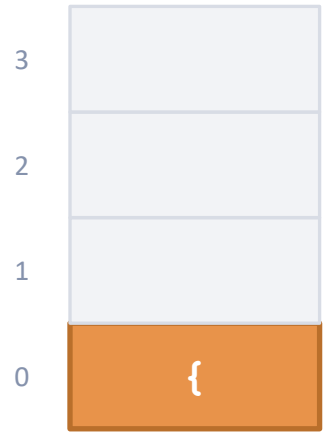
```
int eslesiyor(char ac, char kapa) {
    return (ac=='(' && kapa==')') || (ac=='[' && kapa==']') || (ac=='{' && kapa=='}');
}

// Dönüş: 1=dengeli, 0=dengesiz, -1=HATA (kapasite aşıldı/geçersiz girdi)
int dengeliMi(char *ifade) {
    struct Stack s;  baslat(&s);
    for (int i = 0; ifade[i] != '\0'; i++) {
        char c = ifade[i];
        if (c=='(' || c=='[' || c=='{') {
            if (push(&s, c) == -1) return -1;  // HATA: kapasite aşıldı
        } else if (c==')' || c==']' || c=='}') {
            int ust;
            if (isEmpty(&s) || pop(&s,&ust)==-1 || !eslesiyor((char)ust,c))
                return 0;                          // dengesiz
        }
    }
    return isEmpty(&s) ? 1 : 0;  // 1:dengeli  0:dengesiz
}
```

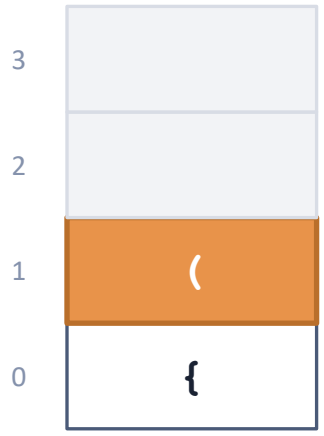
Görsel Örnek — İfade: { ([ ]) }

Adım 1: Açan parantezler push edilir

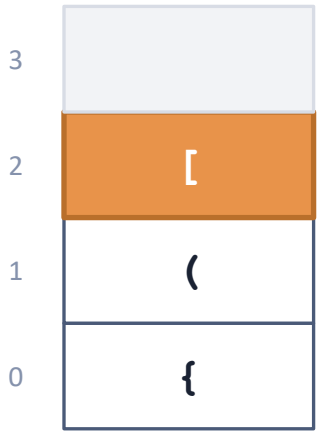
`oku: {`



`oku: (`



`oku: [`



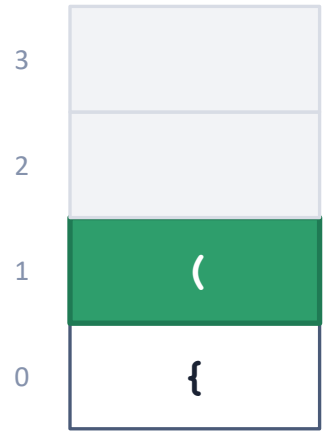
Her açan parantez görüldüğünde, kapasite kontrolü yapılarak stack'e eklenir. Bu örnekte stack'in ulaştığı en büyük derinlik 3'tür.Kopyala

Görsel Örnek — İfade: { ([ ]) }

Adım 2: Kapayan parantezler pop edilip eşleştirilir

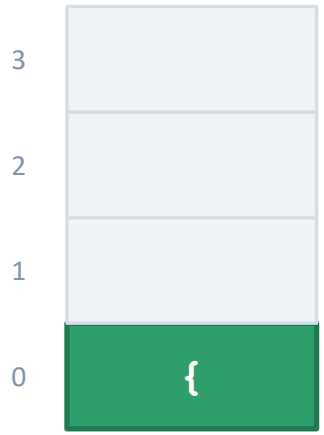
`oku: ]`

pop [→] ile eşleşti ✓



`oku: )`

pop (→) ile eşleşti ✓



`oku: }`

pop { → } ile eşleşti ✓

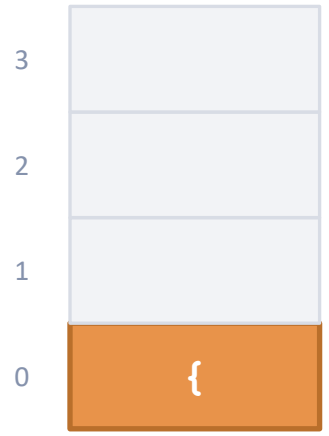
İfade sonuna ulaşıldığında stack BOŞ → tüm parantezler eşleşti → sonuç: DENGELİ.

Görsel Örnek — Dengesiz İfade: { ( ]) }

Hatanın Yakalandığı An

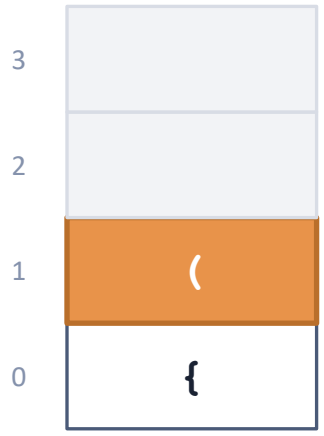
`oku: {`

push {



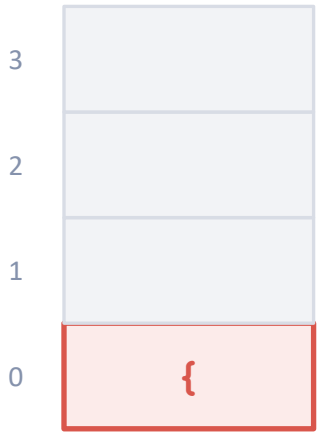
`oku: (`

push (



`oku: ]`

pop (≠] → HATA!



3. karakterde (']') stack'in tepesi '(' — ama ']' bir '[' bekliyor. Eşleşme BAŞARISIZ → algoritma HEMEN durur ve YANLIŞ döndürür. Kalan karakterler hiç okunmaz.

Parantez Kontrolü — Karmaşıklık Analizi

$$T(n) = n \rightarrow O(n)$$

Her karakter TAM OLARAK bir kez okunur; her karakter için push/pop/karşılaştırma O(1)'dir. n karakterlik bir ifade için toplam işlem sayısı n ile doğru orantılıdır. Alan karmaşıklığı da en kötü durumda O(n) (tüm karakterler açan parantezse).

Infix, Postfix, Prefix Nedir?

Uygulama 2

Tür	Örnek	Operatör Konumu
Infix (alışılmış)	`A + B`	İki operand ARASINDA
Postfix (RPN)	`A B +`	İki operanddan SONRA
Prefix (Polish)	`+ A B`	İki operanddan ÖNCE

İnsanlar infix ile düşünür (A + B), ama bilgisayarlar postfix'i çok daha kolay hesaplar — çünkü parantez gerektirmez ve operatör önceliği konusunda BİRDEN belirsiz değildir. Bu haftanın geri kalanı bu iki gözlemi kanıtlayacak.

Neden Postfix'e İhtiyaç Duyulur?

Örnek: A + B * C

Infix `A + B * C`

İşlem sırasını bilmek için operatör önceliği kurallarına İHTİYAÇ VAR

Postfix `A B C * +`

*İşlem sırası DOĞRUDAN OKUNUR: önce B*C, sonra +A*

Postfix'te parantez YOKTUR ve işlem sırası hiçbir kural bilmeden, sadece SOLDAN SAĞA okunarak belirlenir. Bu, bilgisayarlar için ideal bir formattır — stack ile doğrudan hesaplanabilir.

Operatör Önceliği (Precedence) Tablosu

Öncelik	Operatörler	Açıklama
3 (en yüksek)	^	Üs alma
2	* /	Çarpma, bölme
1 (en düşük)	+ -	Toplama, çıkarma

Yüksek öncelikli operatör, düşük öncelikliden ÖNCE işlenir. Örnek: A + B * C ifadesinde * (öncelik 2), + (öncelik 1)'den önce işlenir → sonuç A + (B*C), (A+B)*C DEĞİL.

Birleşme Yönü (Associativity) Kavramı

- **Aynı önceliğe sahip operatörler yan yana geldiğinde, HANGİ YÖNDEN işleneceğini belirler.**

Operatör	Yön	Örnek
+ - * /	Soldan Sağa	A - B - C = (A - B) - C
^ (üs alma)	Sağdan Sola	A ^ B ^ C = A ^ (B ^ C)

Yaygın Hata: A - B - C'yi A - (B - C) sanmak. Doğrusu (A-B)-C'dir — soldan sağa birleşim nedeniyle. Bu fark, postfix dönüşümünde önemli olacak.

Infix → Postfix Dönüşümü — Genel Mantık

- Operandlar (harf/sayı) DOĞRUDAN çıktıya yazılır.
- **Operatörler bir STACK'te BEKLETİLİR; yeni operatör geldiğinde, stack'teki DAHA YÜKSEK öncelikli (veya EŞİT öncelikli VE soldan sağa birleşen) operatörler önce çıktıya AKTARILIR.**
- **İSTİSNA: ^ (üs alma) SAĞDAN SOLA birleşir — eşit öncelikli iki '^' üst üste geldiğinde önceki POP EDİLMEZ.**
- '(' görüldüğünde push edilir; ')' görüldüğünde stack'ten '(' bulunana kadar hepsi çıktıya aktarılır.

Anahtar Fikir: Stack, “bekleyen” operatörleri doğru sırada tutarak, parantezlere ihtiyaç duymadan doğru işlem sırasını üretir.

Infix → Postfix — Pseudocode

```
Fonksiyon InfixToPostfix(infix)
    cikis ← ""
    s ← BoşStack()
    her karakter c için infix'te
        eğer c operand ise (harf/sayı)
            cikis ← cikis + c
        eğer c == '('
            push(s, c)
        eğer c == ')'
            iken peek(s) ≠ '('
                cikis ← cikis + pop(s)
            pop(s)   // '(' atılır
        eğer c operatör ise
            iken !isEmpty(s) ve peek(s)≠'(' ve
                  (öncelik(peek(s)) > öncelik(c) veya
                   (öncelik(peek(s))==öncelik(c) ve
                    c SOLDAN SAĞA birleşiyorsa))
                cikis ← cikis + pop(s)
            push(s, c)
    iken !isEmpty(s)
        cikis ← cikis + pop(s)
    dön cikis
```

Infix → Postfix — C Kodu (1/2)

Öncelik fonksiyonu ve ana döngü

```
int oncelik(char op) {
    if (op=='^') return 3;
    if (op=='*'||op=='/') return 2;
    if (op=='+'||op=='-') return 1;
    return 0;
}
int sagdanSolaBirlesir(char op) { return op=='^'; }   // us alma: sagdan sola

int infixToPostfix(char *infix, char *postfix, int kapasite) {
    struct Stack s;  baslat(&s);  int k = 0;
    for (int i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];
        if (isalnum((unsigned char)c)) {
            if (k >= kapasite-1) return -1;   // cikti tasmasi
            postfix[k++] = c;
        } else if (c == '(') {
            if (push(&s, c) == -1) return -1;  // stack tasmasi
        } else if (c == ')') {
            int top, bulundu = 0;
            while (!isEmpty(&s)) {
                peek(&s, &top);
                if (top == '(') { bulundu = 1; break; }
                pop(&s, &top);
                if (k >= kapasite-1) return -1;
                postfix[k++] = (char)top;
            }
            if (!bulundu) return -1;  // eslesen '(' yok -> gecersiz ifade
            pop(&s, &top);            // '(' atılır
```

Infix → Postfix — C Kodu (2/2)

')' ve operatör durumları, ve son boşaltma

```
        } else if (oncelik(c) > 0) {     // geçerli bir operatör
            int top;
            while (!isEmpty(&s)) {
                peek(&s, &top);
                if (top == '(') break;
                int tO = oncelik((char)top), cO = oncelik(c);
                if (tO < cO) break;
                if (tO == cO && sagdanSolaBirlesir(c)) break;  // İSTİSNA
                pop(&s, &top);
                if (k >= kapasite-1) return -1;
                postfix[k++] = (char)top;
            }
            if (push(&s, c) == -1) return -1;   // stack taşması
        } else return -1;   // tanınmayan/geçersiz karakter
    }
    while (!isEmpty(&s)) {
        int top;  pop(&s, &top);
        if (top == '(') return -1;   // eşleşmeyen '(' kaldı
        if (k >= kapasite-1) return -1;
        postfix[k++] = (char)top;
    }
    postfix[k] = '\0';
    return 0;
}
```

Görsel Örnek — A + B * C - D

Tam İz Tablosu

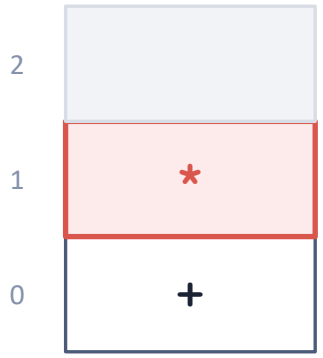
Sembol	İşlem	Stack (taban→tepe)	Çıktı
`A`	operand → çıktıya yaz	(boş)	`A`
`+`	stack boş → push	+	`A`
`B`	operand → çıktıya yaz	+	`AB`
`*`	öncelik(*)>öncelik(+) → pop yok, push	+ *	`AB`
`C`	operand → çıktıya yaz	+ *	`ABC`
`-`	öncelik(*)≥öncelik(-) → pop *; öncelik(+)≥öncelik(-) → pop +; push -	-	`ABC*+`
`D`	operand → çıktıya yaz	-	`ABC*+D`
`(bitiş)`	kalan hepsi pop	(boş)	`ABC*+D-`

Kritik An: '-' Okunduğunda Çift Pop

Öncelik(*) ≥ öncelik(-) VE öncelik(+) ≥ öncelik(-) olduğu için İKİSİ DE çıkar

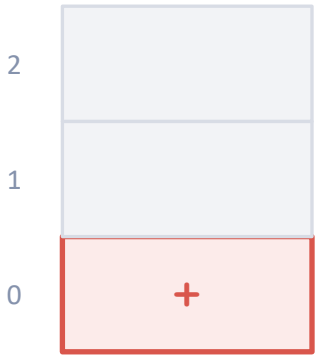
Önce ('-' okunmadan hemen önce)

`çıktı: ABC`



Adım 1: pop *

`çıktı: ABC*`



Adım 2: pop +

`çıktı: ABC*+`

Ardından '-' push edilir. Bu, TEK bir sembol için BİRDEN FAZLA pop olabileceğinin somut kanıtıdır.

Sonuç ve Doğrulama

```
A + B * C - D   →   A B C * + D -
```

Sayısal doğrulama: A=2, B=3, C=4, D=1

Yöntem	Hesap	Sonuç
Infix (alışılmış sıra)	2 + 3×4 - 1 = 2+12-1	13
Postfix (stack ile)	önce 3*4=12, sonra 2+12=14, sonra 14-1	13

İki yöntem AYNI sonucu (13) verir — çünkü postfix dönüşümü, operatör önceliğini ZATEN doğru sıraya kodlamıştır. Şimdi bu postfix ifadeyi stack ile NASIL hesaplayacağımızı göreceğiz.

Postfix Değerlendirme — Pseudocode

```
Fonksiyon PostfixDegerlendir(postfix)
    s ← BoşStack()
    her token t için postfix (boşlukla ayrılmış)
        eğer t sayı ise
            push(s, t)
        değilse   // t operatör ise
            b ← pop(s)      // İKİNCİ (sağdaki) operand — ÖNCE gelir
            a ← pop(s)      // BİRİNCİ (soldaki) operand — SONRA gelir
            sonuc ← a t b   // örn: a - b   (b - a DEĞİL!)
            push(s, sonuc)
    dön pop(s)
```

EN KRİTİK SATIR: b ÖNCE pop edilir (sağ operand), a SONRA pop edilir (sol operand). Çıkarma ve bölmede a-b ile b-a TAMAMEN farklı sonuçlar verir!

Postfix Değerlendirme — C Kodu

```c
// Dönüş: 0=başarılı (sonuç *sonuc'a yazılır), -1=HATA. strtok KULLANMAZ
// (girdiyi değiştirmez); tek karakterli operand/operatörle çalışır.
int postfixDegerlendir(char *postfix, int *sonuc) {
    struct Stack s;  baslat(&s);
    for (int i = 0; postfix[i] != '\0'; i++) {
        char c = postfix[i];
        if (isspace((unsigned char)c)) continue;   // boşluklu da çalışır
        if (isdigit((unsigned char)c)) {
            if (push(&s, c - '0') == -1) return -1;   // stack taşması
        } else if (c=='+'||c=='-'||c=='*'||c=='/') {
            int b, a;
            if (pop(&s, &b) == -1) return -1;  // 2. operand (sağ) — ÖNCE
            if (pop(&s, &a) == -1) return -1;  // 1. operand (sol) — SONRA
            int hesap;
            if (c == '+')      hesap = a + b;
            else if (c == '-') hesap = a - b;   // a-b, b-a DEĞİL
            else if (c == '*') hesap = a * b;
            else {
                if (b == 0) return -1;           // sıfıra bölme
                hesap = a / b;                    // a/b, b/a DEĞİL
            }
            if (push(&s, hesap) == -1) return -1; // stack taşması
        } else return -1;   // tanınmayan karakter
    }
    if (pop(&s, sonuc) == -1) return -1;   // boş ifade/eksik operand
    if (!isEmpty(&s)) return -1;            // fazladan değer kaldı
    return 0;
}
```

Görsel Örnek — 6 2 3 + - 3 8 2 / + *

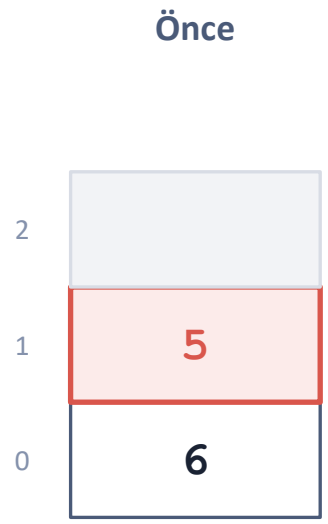
Tam İz Tablosu

Token	İşlem	Stack (taban→tepe)
6	push 6	6
2	push 2	6 2
3	push 3	6 2 3
+	b=3,a=2 → 2+3=5, push	6 5
−	b=5,a=6 → 6-5=1, push	1
3	push 3	1 3
8	push 8	1 3 8
2	push 2	1 3 8 2
/	b=2,a=8 → 8/2=4, push	1 3 4
+	b=4,a=3 → 3+4=7, push	1 7
*	b=7,a=1 → 1*7=7, push	7

Sonuç: stack'te tek eleman kalır → 7. Bu, (6-(2+3))*(3+(8/2)) infix ifadesiyle aynı sonuçtur.

Kritik An 1: '-' İşleniyor

Stack: taban→tepe = 6, 5 (5 tepede, en son push edilen)



DOĞRU

```
b=pop()=5 (sağ)
a=pop()=6 (sol)
a-b = 6-5 = 1
```

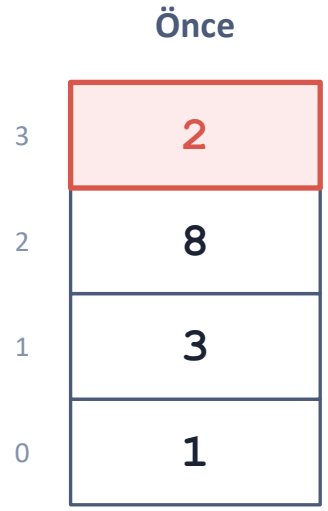
YANLIŞ (sıra ters)

```
a=pop()=5 (yanlış!)
b=pop()=6 (yanlış!)
a-b = 5-6 = -1
```

İlk pop edilen SAĞ operand, ikinci pop edilen SOL operand'dır. Doğru sonuç 1'dir. Bu değer, 54. slayttaki postfix değerlendirme tablosuyla da tutarlıdır.

Kritik An 2: '/' İşleniyor

Stack: taban→tepe = 1, 3, 8, 2 (2 tepede, en son push edilen)



DOĞRU

```
b=pop()=2  (sağ)
a=pop()=8  (sol)
a/b = 8/2 = 4
```

YANLIŞ (sıra ters)

```
a=pop()=2  (yanlış!)
b=pop()=8  (yanlış!)
a/b = 2/8 = 0
```

Bölmede hata DAHA da SİNSİDİR: 2/8, C'de TAM SAYI bölmesiyle 0 verir — program çökmez, sadece SESSİZCE YANLIŞ sonuç üretir. Doğru sıra olmadan bu hatayı fark etmek çok zordur.

Operand Sırası Neden Önemli?

```
ALTIN KURAL: b = pop()  (SAĞ operand),  a = pop()  (SOL operand)  →
                        sonuç = a OP b
```

Operatör	Sıra ÖNEMLİ Mİ?	Neden
+ ve *	Hayır (önemsiz)	a+b = b+a, a*b = b*a (değişmeli)
- ve /	EVET (ÇOK ÖNEMLİ)	a-b ≠ b-a, a/b ≠ b/a (değişmeli DEĞİL)

Bu, bu haftaki tüm postfix hesaplama kodlarında (ve ödevde!) unutulmaması gereken TEK EN ÖNEMLİ kuraltır. + ve * için sıra fark etmez, ama - ve / için mutlaka doğru sırayı kullanmalısınız.

Postfix Değerlendirme — Karmaşıklık Analizi

$$T(n) = n \rightarrow O(n)$$

Her token TAM OLARAK bir kez işlenir; her token için push/pop/hesaplama O(1)'dir. n tokenlık bir postfix ifade için toplam işlem sayısı n ile doğru orantılıdır — tam olarak infix→postfix dönüşümü ile aynı karmaşıklık sınıfı.

Geri Alma (Undo) ve Geri Dönüş (Backtracking)

Geri Alma (Undo)

- Her işlem yapıldığında stack'e push edilir
- Ctrl+Z basıldığında en son işlem pop edilip geri alınır
- Örnek: metin editörleri, çizim programları

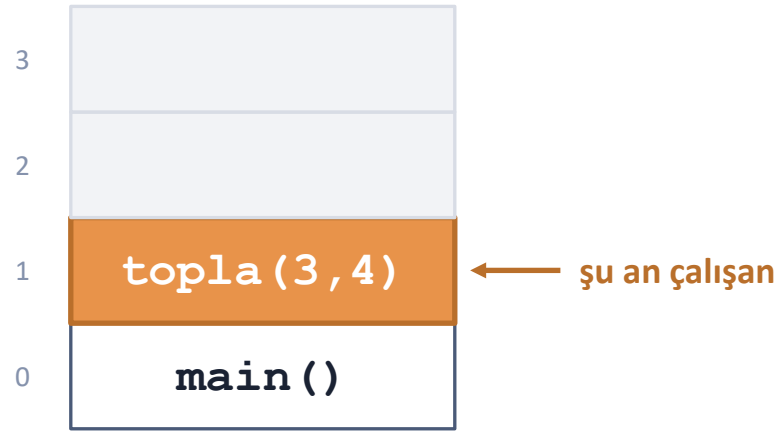
Geri Dönüş (Backtracking)

- Bir yolda ilerlerken çıkmaza girilirse, EN SON karar geri alınır
- Stack, ziyaret edilen konumları/kararları tutar
- Örnek: labirent çözme, bulmaca algoritmaları (Sudoku)

Ortak Nokta: HER İKİSİNDE de LIFO davranışı doğaldır — “en son yapılanı ilk geri al” mantığı, bir stack'in tam olarak yaptığı şeydir.

Fonksiyon Çağrı Yığını (Call Stack)

- Bir fonksiyon başka bir fonksiyonu çağırdığında, çağıran fonksiyonun bilgisi (yerel değişkenler, dönüş adresi) bir STACK'E push edilir.



main() topla(3,4)'ü çağırır → push. topla() bitince POP edilir ve main()'e geri dönülür — tam olarak bir stack'in push/pop davranışı.

Stack ve Özyineleme (Recursion) İlişkisi

5. Haftaya Köprü

- **Özyinelemeli (kendini çağıran) bir fonksiyon, HER çağırıda call stack'e YENİ bir çerçeve (frame) push eder.**
- Taban durum (base case) ulaşıldığında, çerçeveler SIRAYLA pop edilerek geri dönülür.
- **Bu yüzden çok derin özyineleme "stack overflow" hatasına yol açabilir — bu haftaki 12. slaytta gördüğümüz aynı kavram!**

Bu hafta öğrendiğiniz push/pop mantığını aklınızda tutun — 5. hafta, özyinelemeyi tam olarak bu "çerçeve üstüne çerçeve" fikriyle derinlemesine işleyecek.

Hanoi Kuleleri — Kısa Motivasyon

Sadece tanıtım — çözüm 5. haftada

- Problem: 3 çubuk ve farklı boyutlarda diskler var. Tüm diskleri, HER SEFERİNDE TEK disk taşıyarak ve BÜYÜK diski KÜÇÜĞÜN üstüne KOYMADAN, bir çubuktan diğerine taşıyın.
- **Bu problem, DOĞASI GEREĞİ özyinelemelidir — ve çözüm sırasında YİNE bir stack (bu kez call stack) devreye girer.**

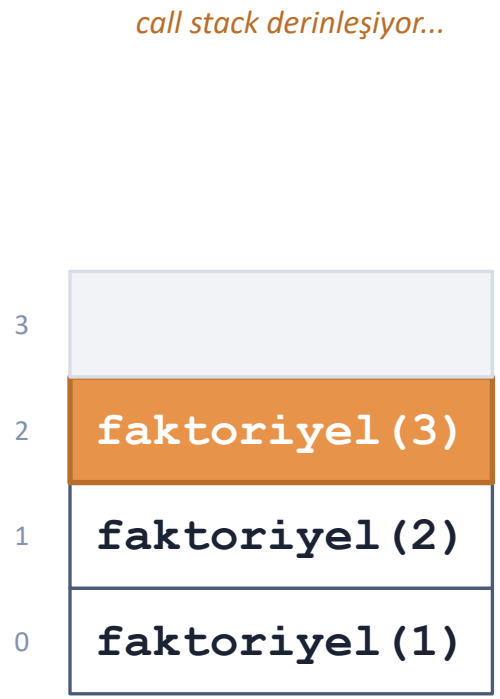
Bu hafta bu problemi ÇÖZMEYECEĞİZ — sadece TANIYORUZ. Tam çözüm, özyineleme mantığını tam olarak işlendikten sonra 5. haftada ele alınacak.

Haftalık Özet

- Stack ADT'yi (push, pop, peek, isEmpty, isFull) ve LIFO prensibini öğrendik.
- Diziyle ve bağlı listeyle stack gerçekleştirdik — ikisi de O(1) işlem verir.
- Dengeli parantez kontrolünü, her karakter için stack durumunu izleyerek uyguladık.
- Infix ifadeleri postfix'e çevirdik ve operatör önceliğini/birleşimini doğru uyguladık.
- Postfix ifadeleri stack ile hesaplarken OPERAND SIRASININ (özellikle çıkarma/bölmede) kritik önemini kanıtladık.
- Stack'in call stack ve özyineleme ile ilişkisini görerek 5. haftaya köprü kurduk.

Gelecek Hafta Önizleme

5. Hafta: Özyineleme (Recursion)



- Özyineleme (recursion) kavramı ve taban durum (base case)
- Faktöriyel, Fibonacci gibi klasik örnekler
- **Bu hafta gördüğümüz call stack kavramı doğrudan kullanılacak**
- Hanoi Kuleleri'nin TAM çözümü
- Özyineleme ile yineleme (iteration) karşılaştırması

Hazırlık: Bu haftaki "stack overflow" ve "call stack" konularını tekrar edin — 5. hafta bunların üzerine inşa edilecek.

Kavrama Soruları / Mini Quiz

1) Boş bir dizi tabanlı stack'te (kapasite=5) 3 push sonrası top değeri kaçtır? Ardından 1 pop sonrası top kaç olur?

2) Bağlı liste tabanlı stack'te push işlemi neden 3. haftadaki 'başa ekleme' ile aynıdır?

3) Postfix '5 3 -' ifadesinin sonucu nedir? '3 5 -' ile aynı sonucu verir mi? Neden?

4) Infix 'A * B + C' ifadesinin postfix karşılığı nedir? Adım adım gösteriniz.

Haftalık Ödev

- 1) Dizi tabanlı bir stack (struct Stack, kapasite=10) tanımlayıp push/pop/peek/isEmpty/isFull fonksiyonlarını C'de yazınız. Overflow/underflow durumlarını test ediniz.
- 2) '{[(A+B)*C]-D}' ifadesinin dengeli olup olmadığını, bu haftaki slayt formatında (her karakter ayrı gösterilerek) elle çözünüz.
- 3) 'A * (B + C) - D / E' infix ifadesini postfix'e çeviriniz — tam iz tablosu (49. slayt formatında) ile gösteriniz.
- **4) Elde ettiğiniz postfix ifadeyi A=2,B=3,C=4,D=8,E=2 değerleriyle stack kullanarak elle hesaplayınız; çıkarma/bölme adımlarında operand sırasını açıkça belirtiniz.**

Teslim: Bir sonraki ders öncesi, C kaynak dosyaları + elle çözülmüş adımları gösteren bir PDF olarak.

Kaynakça

Kaynak	İlgili Bölüm(ler)	Sayfa Aralığı
Horowitz, Sahni, Anderson-Freed — Fundamentals of Data Structures in C	Chapter 3: Stacks and Queues	s. 83–136
Thareja — Data Structures Using C	Chapter 7: Stacks	s. 231–268

Not:

Push/pop görselleri, parantez kontrolü örnekleri, infix→postfix dönüşüm izi ve postfix değerlendirme adım adım örnekleri özgün olarak hazırlanmıştır; kaynaklardaki ilgili bölümler kavramsal çerçeve için referans alınmıştır.